AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Oauth Session

Table of Contents

OAuth And Session Documentation

Google OAuth Strategy

1. Browser opens `/login.html`.
2. React calls `GET /api/v1/auth/config` to read the public Google OAuth client id.
3. Browser obtains a Google ID token through Google Identity Services.
4. Browser sends the ID token to `POST /api/v1/auth/google`.
5. Backend verifies the ID token with Google in production using `google-auth-library`.
6. Backend validates `sub`, `email` and configured audience `GOOGLE_CLIENT_ID`.
7. Backend links `google_sub` to an internal user record.
8. If the user is new, the default internal role is `Student`.
9. Internal roles are assigned only through the application, mainly `PATCH /api/v1/users/{id}/role`.

Google Console Origin Rules

Google OAuth browser origins must use an allowed web origin. For deployed environments, use an HTTPS origin with a public domain name. Do not use a raw public IP such as `http://18.217.255.109`; Google rejects raw IP origins because they do not end in a public top-level domain. Localhost origins such as `http://localhost:5500` and `http://127.0.0.1:5500` are acceptable for local development.

Recommended production origin:

- `https://<owned-domain>`

Temporary testing origin, only after DNS and HTTPS are configured:

- `https://18-217-255-109.sslip.io`

Google documentation reference:
https://support.google.com/cloud/answer/15549257

Test/Development Mode

`ALLOW_MOCK_GOOGLE_TOKENS=true` or `NODE_ENV=test` allows signed mock JWT payloads for automated tests. Production must keep mock tokens disabled.

Session Strategy

- The application issues its own signed JWT session token after Google verification.
- `POST /api/v1/auth/google` returns the token as `data.sessionToken` with `data.tokenType = "Bearer"` for Postman/API verification.
- The same token is also sent in cookie `alc_session` for browser/private page usage.
- Cookie options: HttpOnly, SameSite strict, Secure in production.
- Server persistence stores only a SHA-256 token hash, not the raw token.
- Session expiration defaults to `SESSION_TTL_MINUTES=120`.
- Authenticated API requests can use either cookie `alc_session` or `Authorization: Bearer <sessionToken>`.
- Logout revokes the server-side session hash and clears the cookie; the same Bearer token must return `401` after logout.

Postman Session Verification

1. Run `Auth & Session / Google Login - Real Token`.
2. The Postman test stores `data.sessionToken` into `session_token`.
3. Run `Auth & Session / Current Session - Bearer Token`; it must return `200`.

4. Run `Session Teardown / Logout`; the backend revokes the persisted session hash.
5. Run `Session Teardown / Current Session - After Logout`; the same Bearer token must return `401`.

Back Button And Expiration Controls

- API and private pages send `Cache-Control: no-store`.
- `/private/*` is protected by session middleware.
- Missing/expired session redirects private HTML pages to `/login.html?session=expired`.
- Expired API sessions return `401`.

Frontend Private Entry

- Public landing button `Private system` points to `/login.html`.
- Successful Google login redirects to `/private/dashboard.html`.
- Logout revokes the server-side session and redirects to `/login.html?session=logout`.
- The Content Security Policy allows the Google Identity Services script/frame while keeping application resources self-hosted.

Required Environment Variables

- `GOOGLE_CLIENT_ID`
- `SESSION_SECRET`
- `SESSION_TTL_MINUTES`
- `ALLOW_MOCK_GOOGLE_TOKENS`
- `CORS_ORIGINS`